

Huffman Coding: A Verified Case Study

Lecture 21

Section 1

Part I: The Problem

From theory to practice

In Lectures 19 and 20 we built small types (Bool, Nat, List, Vec) and proved small theorems about them. Today we tackle something larger: a complete **data compression algorithm**, with a proof that decompression recovers the original message.

This is a case study in the style of Stump, Chapter 8. The goal is to see how the techniques from the last two lectures — induction, pattern matching, rewriting — scale to a real problem.

Fixed-length vs. variable-length codes

Suppose we have a message over the alphabet $\{a, b, c, d\}$. A fixed-length code uses 2 bits per symbol:

$$a \mapsto 00, \quad b \mapsto 01, \quad c \mapsto 10, \quad d \mapsto 11$$

The message aab encodes as 00 00 01 (6 bits). Every symbol costs the same, regardless of how often it appears.

A **variable-length code** assigns shorter codes to frequent symbols and longer codes to rare ones. If a is common, we might assign $a \mapsto 0$, saving a bit each time it appears.

The prefix-free condition

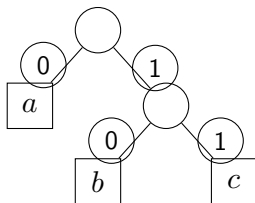
Variable-length codes have a problem: the encoding of ab might look identical to the encoding of c if codes overlap. We need **prefix-freeness**: no code is a prefix of another.

Code	Prefix-free?	Why
$a \mapsto 0, b \mapsto 01$	no	0 is a prefix of 01
$a \mapsto 0, b \mapsto 10, c \mapsto 11$	yes	no code prefixes another

With a prefix-free code, decoding is unambiguous: scan the bits left to right, and at each point there is at most one matching symbol.

Trees give prefix-free codes for free

A binary tree with symbols at the leaves defines a prefix-free code: the code for each symbol is its **path from the root** (left = 0, right = 1).



$$a \mapsto [0], \quad b \mapsto [1, 0], \quad c \mapsto [1, 1]$$

Prefix-freeness is automatic: a leaf path can never be a prefix of another leaf path, because a leaf has no children to extend the path.

The cost of an encoding

Given a code tree t and a message, the **total number of bits** used is:

$$\text{cost}(t) = \sum_{s \in \text{symbols}} f(s) \cdot d_t(s)$$

where $f(s)$ is the number of times symbol s appears in the message, and $d_t(s)$ is the **depth** of s in tree t (the number of edges from root to leaf = the code length).

For example, with the tree on the previous slide ($a \mapsto [0]$, $b \mapsto [1, 0]$, $c \mapsto [1, 1]$) and a message containing 5 a 's, 2 b 's, and 1 c :

$$\text{cost} = 5 \cdot 1 + 2 \cdot 2 + 1 \cdot 2 = 11 \text{ bits}$$

A tree is **optimal** for given frequencies if no other tree over the same symbols achieves a lower cost.

Huffman's algorithm: the idea

David Huffman (1952) gave a greedy algorithm that, given symbol frequencies, constructs a tree of **minimum cost** for those frequencies. It works with a collection of **weighted trees**:

A **weighted tree** is a pair (w, t) where w is a natural number (the weight) and t is a code tree. We start with one weighted tree per symbol: if symbol s has frequency $f(s)$, we create the weighted tree $(f(s), \text{leaf } s)$.

To **merge** two weighted trees (w_1, t_1) and (w_2, t_2) : create a new node with t_1 on the left and t_2 on the right, and assign it weight $w_1 + w_2$:

$$(w_1, t_1) \oplus (w_2, t_2) = (w_1 + w_2, \text{node } t_1 \ t_2)$$

Huffman's algorithm: the steps

- 1 **Initialize:** for each symbol s with frequency $f(s)$, create the weighted tree $(f(s), \text{leaf } s)$. Collect these into a list.
- 2 **Repeat:** pick the two weighted trees with the **smallest weights**, merge them, and put the result back. The list shrinks by one each step.
- 3 **Stop:** when one weighted tree (w, t) remains. Return t .

Why this produces a minimum-cost tree

Key fact: in any optimal tree, the two symbols with the lowest frequencies are **siblings** (children of the same node) at the maximum depth.

Why: suppose symbol a has the lowest frequency and sits at depth d . If another symbol b sits deeper, at depth $d' > d$, then swapping them changes the cost by $(f(a) - f(b))(d' - d)$. Since $f(a) \leq f(b)$ and $d' > d$, this quantity is ≤ 0 : the cost does not increase. So we can always push the lowest-frequency symbols to the bottom. Repeating for the second-lowest makes them siblings.

Consequence: merging the two lightest symbols first is safe — we are committing to a structure that some optimal tree already has. After the merge, the combined weight $f(a) + f(b)$ correctly accounts for both symbols sitting one level deeper. The remaining problem has one fewer symbol, and the same argument applies recursively.

We will not formalize this proof. Our verification focuses on a different property: that encode and decode are inverses.

Plan for today

Today we will:

- 1 Define the tree type and the encode/decode functions.
- 2 **Prove** that decoding an encoded message recovers the original.
- 3 Implement the tree-building algorithm.

The proof of correctness (item 2) is the main event.

Transitioning to the standard library

In Lectures 19 and 20 we built our own `Bool`, `N`, `List`, and `Vec` from scratch. From now on, we use Lean's standard library: `Nat`, `List`, `Bool`, `Option`.

These are the same inductive types we defined, but the library provides additional infrastructure: notation (`+`, `::`, `++`), decision procedures (`DecidableEq`), and tactics (`omega`, `simp`). We will introduce these as they arise.

Section 2

Part II: The Code Tree

Bits and trees

```
inductive Bit : Type where
  | 0 : Bit      -- left / zero
  | 1 : Bit      -- right / one
deriving DecidableEq

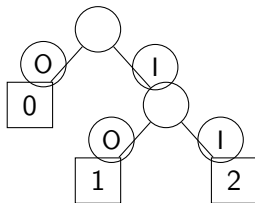
inductive HTree : Type where
  | leaf (sym : Nat) : HTree
  | node (left right : HTree) : HTree
```

Bit has two constructors. HTree is a binary tree with natural numbers at the leaves — each natural number represents a symbol we want to encode (in a real system, these would be characters or bytes; we use Nat for simplicity). Internal nodes carry no data — they are pure branching points.

The line `deriving DecidableEq` asks Lean to generate a decision procedure for equality on Bit. We will use this later.

An example tree

```
def exTree : HTree :=  
  .node (.leaf 0)  
    (.node (.leaf 1) (.leaf 2))  
-- Codes: 0 → [0], 1 → [I,0], 2 → [I,I]
```



We use 0, 1, 2 for symbols. In a real system these would be characters; the algorithm is the same.

Encoding a single symbol

Given a tree and a symbol, find the path from root to leaf:

```
def encodeSym : HTree → Nat → Option (List Bit)
| .leaf s,    n ⇒
    if n = s then some [] else none
| .node l r,  n ⇒
    match encodeSym l n with
    | some bs ⇒ some (.0 :: bs)
    | none    ⇒
        match encodeSym r n with
        | some bs ⇒ some (.1 :: bs)
        | none    ⇒ none
```

At a leaf: if the symbol matches, the code is the empty list (we have arrived). At a node: try the left subtree; if found, prepend 0. Otherwise try right and prepend 1. If neither, return none.

Aside: if with decidable equality

The expression `if n = s then ... else ...` requires Lean to decide whether $n = s$ at runtime. This works because `Nat` has an instance of `DecidableEq`: a function that, for any n and s , produces either a proof of $n = s$ or a proof of $n \neq s$.

In the `then` branch, we have a proof $h : n = s$. In the `else` branch, we have $h : n \neq s$. Lean generates these automatically; we will use them in our proofs.

We will discuss the mechanism behind `DecidableEq` — **type classes** — in a later lecture. For now, it suffices to know that it lets us branch on equality and get proofs in each branch.

Encoding a message

```
def encodeMsg : HTree → List Nat → Option (List Bit)
| _, []      ⇒ some []
| t, n :: ns ⇒
  match encodeSym t n with
  | none      ⇒ none
  | some bs   ⇒
    match encodeMsg t ns with
    | none      ⇒ none
    | some rest ⇒ some (bs ++ rest)
```

Encode each symbol, then concatenate the bit sequences. If any symbol is not in the tree, the whole encoding fails (returns none).

Seeing it work

```
#eval encodeSym exTree 0      -- some [Bit.0]
#eval encodeSym exTree 1      -- some [Bit.I, Bit.0]
#eval encodeSym exTree 2      -- some [Bit.I, Bit.I]
#eval encodeSym exTree 99     -- none

#eval encodeMsg exTree [0, 1, 2]
-- some [Bit.0, Bit.I, Bit.0, Bit.I, Bit.I]
```

The message $[0, 1, 2]$ encodes as $[0, 1, 0, 1, 1]$: 5 bits for 3 symbols. A fixed-length code would need 6 bits.

Section 3

Part III: Decoding

Decoding one symbol

To decode, walk the tree following the bits. When you reach a leaf, emit the symbol:

```
def decodeOne
  : HTree → List Bit → Option (Nat × List Bit)
| .leaf s,    bs      ⇒ some (s, bs)
| .node _ _,   []      ⇒ none
| .node l _,  .0 :: bs ⇒ decodeOne l bs
| .node _ r,  .1 :: bs ⇒ decodeOne r bs
```

The return type is `Option (Nat × List Bit)`: the decoded symbol *and* the remaining bits. If the bits run out before we reach a leaf, return `none`.

Decoding a full message

We decode a fixed number of symbols. In practice, the message length is stored alongside the compressed data.

```
def decodeMsg
  : HTree → Nat → List Bit → Option (List Nat)
| _, 0, _ ⇒ some []
| t, k + 1, bs ⇒
  match decodeOne t bs with
  | none ⇒ none
  | some (n, rest) ⇒
    match decodeMsg t k rest with
    | none ⇒ none
    | some ns ⇒ some (n :: ns)
```

This is structurally recursive on k : each call decreases the count by one. No termination proof is needed.

Seeing decoding work

```
#eval decodeOne exTree [.0, .I, .0, .I, .I]
-- some (0, [Bit.I, Bit.0, Bit.I, Bit.I])

#eval decodeMsg exTree 3 [.0, .I, .0, .I, .I]
-- some [0, 1, 2]
```

The first call decodes symbol 0 (following 0 to the left leaf) and returns the remaining 4 bits. The full decode recovers [0, 1, 2].

But does this *always* work? Can we **prove** that decoding an encoded message recovers the original? This is the main theorem of the lecture.

Section 4

Part IV: Verification

The roundtrip property

Theorem (roundtrip). *If encoding succeeds, decoding recovers the original message.*

```
theorem roundtrip (t : HTree) (ns : List Nat)
  (bs : List Bit) (h : encodeMsg t ns = some bs)
  : decodeMsg t ns.length bs = some ns
```

The precondition $\text{encodeMsg } t \ ns = \text{some } bs$ says: every symbol in the message is in the tree, and bs is the resulting bit sequence. The conclusion says: decoding bs (asking for $|ns|$ symbols) produces exactly ns .

The proof has two layers: a **key lemma** about single symbols, and the full theorem by induction on the message.

The key lemma

Lemma (decodeOne after encodeSym). *If a symbol encodes to bits bs, then decoding bs followed by any remaining bits recovers the symbol and leaves the remaining bits untouched.*

```
theorem decodeOne_encodeSym (t : HTree) (n : Nat)
  (bs rest : List Bit)
  (h : encodeSym t n = some bs)
  : decodeOne t (bs ++ rest) = some (n, rest)
```

This is where prefix-freeness is used: the bits for one symbol lead to exactly one leaf, and no further bits are consumed.

Proof of the key lemma: setup

We prove `decodeOne_encodeSym` by induction on the tree t , generalizing the bit sequence bs (since each recursive call encodes into a different sub-sequence):

```
theorem decodeOne_encodeSym (t : HTree) (n : Nat)
  (bs rest : List Bit)
  (h : encodeSym t n = some bs)
  : decodeOne t (bs ++ rest) = some (n, rest) := by
induction t generalizing bs with
| leaf s ⇒ ...
| node l r ih_l ih_r ⇒ ...
```

The hypothesis h records that encoding symbol n in tree t produced the bit sequence bs . We must show that decoding $bs ++ rest$ from the root of t returns $(n, rest)$.

Proof of the key lemma: leaf case

In the leaf case, $t = \text{leaf } s$. After unfolding `encodeSym`, the hypothesis h becomes (if $n = s$ then some `[]` else none) = some bs . We split on the `if`:

```
| .leaf s =>
  simp only [encodeSym] at h
  split at h
  • -- branch:  $n = s$ , so  $h : \text{some } [] = \text{some } bs$ 
    -- hence  $bs = []$ , and the goal reduces to:
    --  $\text{decodeOne } (\text{leaf } s) ([] ++ \text{rest}) = \text{some } (n, \text{rest})$ 
    -- which is:  $\text{some } (s, \text{rest}) = \text{some } (n, \text{rest})$ 
    simp_all [decodeOne]
  • -- branch:  $n \neq s$ , so  $h : \text{none} = \text{some } bs$ 
    contradiction
```

In the `then` branch, $n = s$ and $bs = []$. Unfolding `decodeOne` at a leaf gives $\text{some } (s, \text{rest})$, and since $n = s$ this equals $\text{some } (n, \text{rest})$. In the `else` branch, the hypothesis says $\text{none} = \text{some } bs$, which is a contradiction.

Proof of the key lemma: node case (idea)

In the node case, $t = \text{node } l \ r$. After unfolding, h contains a `match` on `encodeSym l n`. We use `split` at h to case-split:

Case 1: `encodeSym l n = some bsl`. Then the full encoding returned `O :: bsl`, so $bs = O :: bs_l$. The goal becomes:

$$\text{decodeOne } (\text{node } l \ r) \ ([O] ++ bs_l ++ rest) = \text{some } (n, rest)$$

The ι -step sees `O` and recurses left: $= \text{decodeOne } l \ (bs_l ++ rest)$. The induction hypothesis `ih_1` (applied to bs_l and the fact `encodeSym l n = some bsl`) closes the goal.

Case 2: `encodeSym l n = none`. Then a nested `match` tries the right subtree — symmetric to Case 1, using `ih_r`.

Case 3: both subtrees return `none`. Then h says `none = some bs`: contradiction.

Proof of the key lemma: node case (tactics)

```
| .node l r ih_l ih_r =>
  simp only [encodeSym] at h
  split at h
  • -- Case 1: encodeSym l n = some bs_l
    next bs_l h_l =>
      simp only [Option.some.injEq] at h; subst h
      simp only [List.cons_append, decodeOne]
      exact ih_l bs_l h_l
  • -- Cases 2 and 3: encodeSym l n = none
    split at h
    • next bs_r h_r =>
      simp only [Option.some.injEq] at h; subst h
      simp only [List.cons_append, decodeOne]
      exact ih_r bs_r h_r
    • contradiction
```

After each `split at h`, the tactic `next` names the new variables: `bs_l` is the bit sequence that `encodeSym l n` returned, and `h_l` is the proof that `encodeSym l n = some bs_l`. These are exactly what `ih_l` needs.

The key lemma: what it says

Stepping back: the lemma says that `decodeOne` is a **left inverse** of `encodeSym`, in the following sense. If encoding symbol n in tree t produces bits bs , then feeding $bs ++ rest$ to `decodeOne` returns $(n, rest)$.

The *rest* bits pass through untouched — the decoder consumes exactly the bits it needs. This is the operational meaning of prefix-freeness.

With this lemma, the full roundtrip is straightforward.

Proof of the roundtrip: base case

We proceed by induction on ns , generalizing bs . Why? In the step case, the original bs is $bs_1 ++ bs_2$, but the IH must apply to bs_2 alone. Without generalizing bs , the IH would be stuck with the original bs .

```
theorem roundtrip (t : HTree) (ns : List Nat)
  (bs : List Bit) (h : encodeMsg t ns = some bs)
  : decodeMsg t ns.length bs = some ns := by
  induction ns generalizing bs with
  | nil =>
    -- h : encodeMsg t [] = some bs
    -- unfolds to: some [] = some bs, so bs = []
    simp [encodeMsg] at h; subst h
    -- goal: decodeMsg t 0 [] = some []
    simp [decodeMsg]
```

The base case: encoding the empty message produces `[]`. Decoding 0 symbols from `[]` gives some `[]`.

Proof of the roundtrip: step case (setup)

```
| cons n ns ih ⇒  
  -- h : encodeMsg t (n :: ns) = some bs  
  -- unfold encodeMsg to extract the pieces:  
  simp only [encodeMsg] at h  
  -- h involves: match encodeSym t n, encodeMsg t ns  
  -- we need to extract: encodeSym t n = some bs1  
  --                      and encodeMsg t ns = some bs2  
  --                      and bs = bs1 ++ bs2
```

The encoding of $n :: ns$ matches on $\text{encodeSym } t \ n$ and $\text{encodeMsg } t \ ns$.
We case-split to extract the pieces.

Proof of the roundtrip: step case (core)

```
split at h
• contradiction -- encodeSym t n = none
• next bs1 h_enc1 ⇒
  split at h
  • contradiction -- encodeMsg t ns = none
  • next bs2 h_enc2 ⇒
    -- h : some (bs1 ++ bs2) = some bs
    -- so bs = bs1 ++ bs2
    simp at h; subst h
    -- goal: decodeMsg t (ns.length + 1)
    --          (bs1 ++ bs2) = some (n :: ns)
    simp only [decodeMsg]
    -- after iota: match decodeOne t (bs1 ++ bs2)
    rw [decodeOne_encodeSym t n bs1 bs2 h_enc1]
    -- goal: ... decodeMsg t ns.length bs2 ...
    --          = some (n :: ns)
    simp [ih bs2 h_enc2]
```

What happened

- 1 **Extract the pieces:** encoding $n :: ns$ means $\text{encodeSym } t \ n = \text{some } bs_1$ and $\text{encodeMsg } t \ ns = \text{some } bs_2$ and $bs = bs_1 ++ bs_2$.
- 2 **Apply the key lemma:** $\text{decodeOne } t \ (bs_1 ++ bs_2) = \text{some } (n, bs_2)$.
- 3 **Apply the IH:** $\text{decodeMsg } t \ |ns| \ bs_2 = \text{some } ns$.
- 4 **Assemble:** the result is some $(n :: ns)$. □

The proof structure mirrors the program structure: encoding processes the head then the tail, so the proof handles the head (key lemma) then the tail (IH).

What the roundtrip theorem guarantees

The theorem says: **if every symbol is in the tree, decoding recovers the original message.** The precondition $\text{encodeMsg } t \text{ } ns = \text{some } bs$ encodes both requirements: that every symbol has a code, and that bs is the correct encoding.

What it does **not** say:

- It does not say the tree is optimal (shortest total encoding). That is a property of the tree-building algorithm, not of encode/decode.
- It does not say what happens if bs is corrupted. We only guarantee the roundtrip for a validly encoded message.

These are separate verification tasks. Correctness of encode/decode is the essential one.

Section 5

Part V: Building the Tree

The algorithm

We now implement Huffman's algorithm from Part I. Recall: given a list of (symbol, frequency) pairs, we repeatedly merge the two lightest weighted trees until one remains.

We need two pieces:

- 1 A **priority queue** of weighted trees (w, t) , sorted by weight w . We implement this as a sorted list with an insertion function.
- 2 A **merge loop** that extracts the two smallest, merges them, and re-inserts until one tree remains.

Weighted trees and sorted insertion

We represent the priority queue as a sorted list of (weight, tree) pairs:

```
def insertSorted
  : (Nat × HTree) → List (Nat × HTree)
    → List (Nat × HTree)
| x, []      ⇒ [x]
| x, y :: ys ⇒
  if x.1 < y.1
  then x :: y :: ys
  else y :: insertSorted x ys
```

This inserts a weighted tree into the list, maintaining ascending order by weight. The comparison `x.1 < y.1` compares the weights.

Building the tree

```
def buildTree : List (Nat × HTree) → Option HTree
| [] ⇒ none
| [(_, t)] ⇒ some t
| (w1, t1) :: (w2, t2) :: rest ⇒
    buildTree
      (insertSorted (w1 + w2, .node t1 t2) rest)
decreasing_by
  simp_all [insertSorted]; omega
```

Each step removes two elements and inserts one, so the list shrinks. The `decreasing_by` block asks Lean to verify this. (The proof requires a lemma that `insertSorted` preserves the list length; we omit the details here.)

Note: this is a **non-structural** recursion — the recursive call is on `insertSorted (...) rest`, not on a direct subterm. Lean accepts it because we provide a termination measure (list length). We will discuss well-founded recursion properly in a later lecture.

From frequencies to a tree

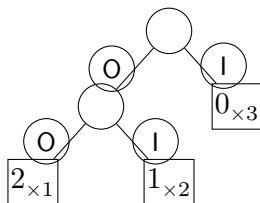
```
def freqsToQueue : List (Nat × Nat)
  → List (Nat × HTree)
| []           ⇒ []
| (sym, freq) :: rest ⇒
    insertSorted (freq, .leaf sym)
    (freqsToQueue rest)

def huffman (fs : List (Nat × Nat)) : Option HTree :=
  buildTree (freqsToQueue fs)
```

The input is a list of (symbol, frequency) pairs. For each pair, we create a leaf and insert it (sorted by frequency) into the queue. Then we run the greedy merge.

Example

```
#eval huffman [(0, 3), (1, 2), (2, 1)]  
-- some (HTree.node  
--       (HTree.node (HTree.leaf 2) (HTree.leaf 1))  
--       (HTree.leaf 0))
```



Symbol 0 (the most frequent) gets a 1-bit code [1]; symbols 1 and 2 get 2-bit codes [0,1] and [0,0].

End-to-end

```
-- Build a tree from frequencies
def myTree := huffman [(0, 3), (1, 2), (2, 1)]

-- Encode and decode
#eval do
  let t ← myTree
  let bs ← encodeMsg t [0, 0, 1, 2, 0]
  -- bs = [I, I, 0, I, 0, 0, I] (7 bits vs 10 fixed)
  let ns ← decodeMsg t 5 bs
  return ns
-- some [0, 0, 1, 2, 0]
```

The roundtrip theorem guarantees this works for **any** tree and **any** message whose symbols are in the tree — not just this example.

Section 6

Part VI: Discussion

What we verified

Property	Status	How
$\text{decode} \circ \text{encode} = \text{id}$	proved	induction on $\text{msg} + t$
Prefix-freeness	proved (implicitly)	follows from tree structure
Termination of encode/decode	checked	structural recursion
Optimality of tree	not verified	would need Kraft's inequality
Termination of buildTree	partially	well-founded measure

The roundtrip theorem is the most important property for correctness. Optimality is about compression quality, not correctness — a sub-optimal tree still encodes and decodes correctly.

The proof method, revisited

The proof followed the same pattern as Lectures 19 and 20:

- ➊ **Identify the recursion:** `encodeSym` recurs on the tree; `encodeMsg` recurs on the list.
- ➋ **Prove a lemma per recursive function:** `decodeOne_encodeSym` follows the recursion of `encodeSym`; `roundtrip` follows the recursion of `encodeMsg`.
- ➌ **Base case:** definitional equality (`rfl` or `simp`).
- ➍ **Step case:** unfold, apply the lemma or IH, simplify.

The program structure determined the proof structure. This is the recurring theme.

What we did not need

The proof did **not** use internal verification. The tree carries no invariant in its type; the proofs are external. We could internalize:

- **Membership:** index the tree by a set of symbols (like BSTM in Lecture 20), so `encodeSym` returns `List Bit` instead of `Option (List Bit)`.
- **Balance:** track the tree depth or weight in the type.

But for this problem, external verification was sufficient and simpler. The art is choosing the right level of verification for the task.

Two things to revisit

Today's lecture used two mechanisms we have not yet explained:

- **Type classes** (`DecidableEq`, `BEq`, `LE`): we used deriving `DecidableEq` and `if n = s then ...` without explaining how Lean resolves these. Type classes are Lean's mechanism for ad-hoc polymorphism — they let us write generic code that works for any type with the right operations.
- **Well-founded recursion**: `buildTree` recurs on a non-subterm. We provided a `decreasing_by` hint, but did not explain the general mechanism. Lean supports recursion on any well-founded relation, not just structural subterms.

These are topics for coming lectures.

Exercises

- 1 Prove that `insertSorted` preserves the list length:
 $|\text{insertSorted } x \text{ } ys| = |ys| + 1.$
- 2 Define a membership predicate `HTree.mem : Nat → HTree → Prop` and prove: if `encodeSym t n = some bs`, then `HTree.mem n t`.
- 3 (Harder) Prove the converse: if `HTree.mem n t`, then `encodeSym t n ≠ none`.
- 4 (Harder) Generalize from `Nat` to any type with `DecidableEq`. What changes in the proofs?
- 5 (Open-ended) What would it take to prove that the Huffman tree is optimal?

All exercise stubs are in the companion Lean file.

What we covered

- ① **The problem:** variable-length, prefix-free codes.
- ② **The code tree:** HTree with encode/decode as tree traversals.
- ③ **The roundtrip proof:** decode after encode recovers the original.
Two-layer proof: a key lemma per symbol, then induction on the message.
- ④ **The algorithm:** Huffman's greedy merge, with sorted insertion as a priority queue.
- ⑤ **Discussion:** what we verified, what we skipped, and what comes next.

Next lecture: type classes and proof automation.

How `DecidableEq`, `BEq`, and `Ord` work under the hood. How Lean's elaborator resolves overloaded notation. And how tactics like `simp`, `omega`, and `decide` automate the kind of proofs we have been writing by hand.